

RCA / Postmortem — Incident #2: Database Connectivity Loss

Field	Value
Incident ID	INC-002
Title	Notes API returning HTTP 503 on all DB-dependent endpoints during PostgreSQL outage
Severity	SEV-2 (major functionality unavailable; no data loss; app process healthy)
Status	Resolved
Author	Vishnu K
Date	2026-08-02
Simulated?	Yes — controlled failure injection for resilience testing
Primary alert	NotesApiHighErrorRate

1. Summary

The PostgreSQL Deployment backing the Notes API was scaled to **zero replicas**, simulating a complete database outage (e.g. a crash, an operator error, or a node eviction). With the database unreachable, every request to the DB-dependent endpoints (`GET/POST /api/notes`) returned **HTTP 503 "database unavailable"**.

Importantly, the application **degraded gracefully**: the app pods stayed `1/1 Running` and continued returning `200` on `/health` throughout the outage. The app logged the connection failure and returned a clean 503 rather than crashing — meaning it was ready to serve the instant the database returned.

The incident was detected by the `NotesApiHighErrorRate` alert and resolved by scaling PostgreSQL back to one replica.

2. Impact

- **User-facing:** All note listing and creation failed with HTTP 503 for the duration of the outage (100% failure rate on DB-dependent endpoints).
 - **Not affected:** The application process stayed up; `/health` returned 200 throughout (probes do not exercise the database).
 - **Data:** No data loss. Existing notes remained intact on the persistent volume; they were served correctly immediately after recovery.
 - **Duration:** ~6 min 20 sec (injection to verified recovery).
-

3. Timeline (UTC)

Time	Event
22:36:18	T+0 — Fault injected: <code>kubectl scale deployment/notes-api-postgres --replicas=0</code>
22:36:29	Load loop started; first HTTP 503 returned immediately
22:36:30	<code>NotesApiHighErrorRate</code> enters Pending (5xx rate above 0.2 threshold)
22:38:30	Alert transitions to Firing — time-to-detection: ~2m 12s
22:42:25	Recovery initiated (<code>--recover</code> scales Postgres back to 1)
22:42:38	PostgreSQL pod Ready; <code>/api/notes</code> returns 200 again

4. Detection

- **Primary alert:** `NotesApiHighErrorRate` — fires when the rate of 5xx responses over a 5-minute window exceeds 0.2 req/s, sustained for 2 minutes. Value at firing: **1.599 req/s** — roughly **8x the threshold**, so detection was unambiguous.
- **Supporting signals:**
 - Grafana "Request Rate by Status" showed a clear spike in 503 responses.
 - Loki captured the exact application error (see below).
- **Time-to-detection:** ~2 min 12 sec.

Symptom-based vs cause-based detection (a deliberate discussion)

`NotesApiHighErrorRate` is a **symptom-based** alert — it detects that *users are seeing 5xx errors*, regardless of the underlying cause. A **cause-based** alert (e.g. "Postgres replica count == 0") would fire faster and pinpoint the database directly.

Both have their place:

- **Symptom-based** catches this DB outage *and* any other failure class that produces 5xx (bad deploy, code bug, dependency timeout). Broad coverage.
- **Cause-based** is faster and more precise but only catches the specific cause it was written for.

A mature setup uses both: symptom alerts for coverage, cause alerts for speed and precise triage. This is captured as a corrective action.

Triage signal in the logs

Loki captured:

```
[ERROR] notes-api - Failed to fetch notes: connection to server at
"notes-api-postgres" (10.42.240.40), port 5432 failed: Connection refused
```

"Connection refused" (not "timed out") is a valuable triage signal: it means nothing was listening on port 5432 at all — i.e. the Postgres pod was gone (scaled to 0), **not** a network partition where packets are silently dropped (which would manifest as a timeout). This distinction directs an on-call engineer straight to "the DB process is down" rather than "investigate the network".

5. Root Cause Analysis (5 Whys)

- **Why did users get errors?** → The Notes API returned HTTP 503 on `/api/notes`.
- **Why did it return 503?** → The app could not connect to PostgreSQL (connection refused on port 5432).
- **Why was the connection refused?** → The PostgreSQL pod was not running (Deployment scaled to 0 replicas).
- **Why did a single scaling action take the whole feature down?** → PostgreSQL runs as a single-replica Deployment with no standby.
- **Why is there no standby / failover?** → The database tier has no HA design: single replica, no read replica, no point-in-time recovery. ← **Root cause: the database is a single point of failure (SPOF) in the current architecture.**

Contributing factor: the app's liveness/readiness probes only check `/health`, which does not exercise the database. So Kubernetes considered the app "ready" and kept routing traffic to it even though its critical dependency was down. This is a reasonable default (the app *is* alive and *does* recover), but it means the platform layer had no independent signal of the DB dependency being broken.

6. What Went Well

- **Graceful degradation.** The app returned a clean 503 and kept its liveness probe green instead of crashing. This is by design — DB calls are wrapped in error handling with a 5-second connection timeout.
 - **Zero-touch recovery.** Once the database returned, the app reconnected on the next request with no restart or manual intervention.
 - **Overwhelming, unambiguous signal.** The 5xx rate was ~8× threshold, and the "connection refused" log made triage immediate.
 - **No data loss.** The PVC preserved all notes; they were served correctly post-recovery (verified via `curl /api/notes`).
-

7. What Could Be Improved

- **Database is a SPOF** — no HA, no replica, no automated failover.
 - **No cause-based DB alert** — detection was purely symptom-driven.
 - **Probes don't reflect the DB dependency** — traffic kept routing to pods that could not fulfil requests. A DB-aware readiness check would let Kubernetes stop sending doomed requests.
 - **Recovery was manual** (scale back up) — no automated remediation.
-

8. Corrective Actions

#	Action	Owner	Priority	Target Date
1	Document DB-recovery steps in the runbook	Vishnu K	High	2026-08-04

#	Action	Owner	Priority	Target Date
2	Add a cause-based alert (NotesApiDatabaseDown) on Postgres replica count / availability for faster, precise detection	Vishnu K	Medium	Backlog
3	Add an optional DB-aware readiness gate so pods report NotReady when the DB is unreachable (stops routing doomed traffic)	Vishnu K	Medium	Backlog
4	For production: use managed or replicated PostgreSQL with automated failover + PITR backups	Platform team	High	Backlog

9. Evidence

Injection & 503 responses

[illegible]

notes-api.rules

/etc/prometheus/rules/prometheus-monitoring-kube-prometheus-prometheus-rulesfiles-0/default-notes-api-alerts-be367e11-a1e4-4990-a9da-0506648e3140.yaml

FIRING (1)

INACTIVE (4)

NotesApiHighErrorRate

FIRING (1)

```
sum(rate(flask_http_request_total(status=~"5..")[5m])) > 0.2
```

for: 2m

app="notes-api" severity="warning"

description

Notes API is returning 5xx errors above threshold for 2+ minutes.

summary

Notes API high 5xx error rate

Alert labels

State

Active Since

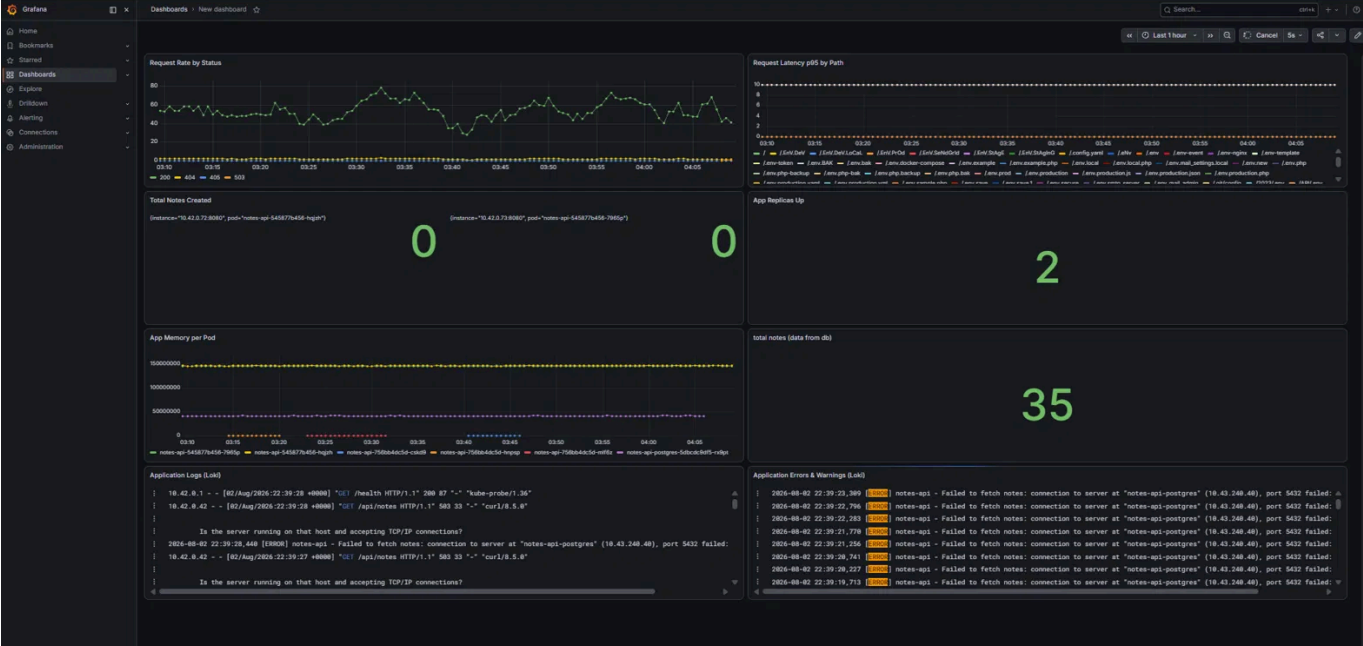
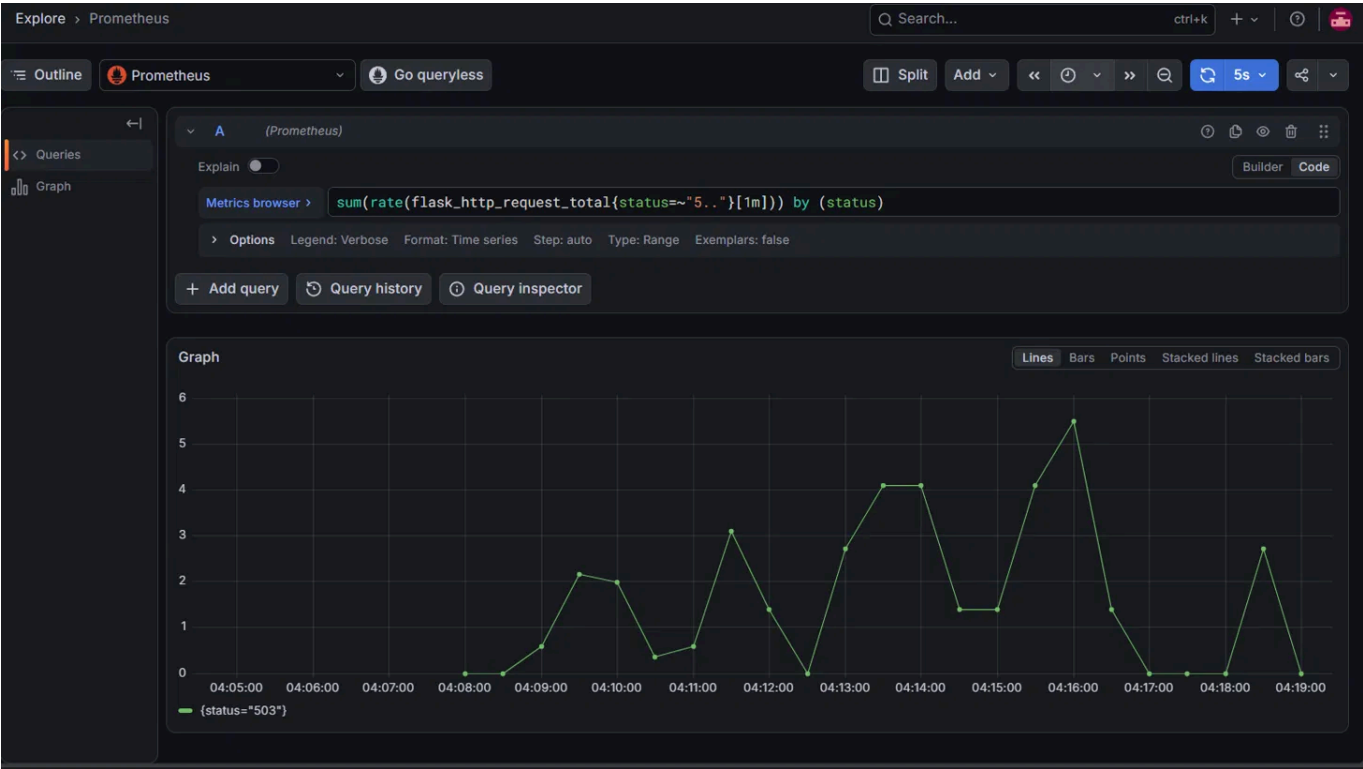
Value

alertname="NotesApiHighErrorRate" app="notes-api" severity="warning"

FIRING

3m 20.997s

1.5990917671957674



Logs (Loki)

Logs

1000 Lines shown - 14.22% (8min 32sec) of 1h Total bytes processed: 818 kB Common labels: namespace=default node=ip-172-31-33-133

Search fields by name

Show log level

Fields

- app 100%
- container 100%
- filename 100%
- job 100%
- namespace 100%
- node_name 100%
- pod 100%
- stream 100%

Expanded

Scroll to bottom

Newest logs first

Search logs

Deduplication

Filter levels

Display ms

Wrap JSON

Columns not supported

Highlight text

Large font

Download logs

Scroll to top

2026-08-03 04:10:20.799 10.42.0.42 -- [02/Aug/2026:22:40:20 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:20.798 2026-08-02 22:40:20.798 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:20.286 10.42.0.42 -- [02/Aug/2026:22:40:20 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:20.285 2026-08-02 22:40:20.285 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:19.773 10.42.0.42 -- [02/Aug/2026:22:40:19 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:19.772 2026-08-02 22:40:19.772 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:19.259 10.42.0.42 -- [02/Aug/2026:22:40:19 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:19.258 2026-08-02 22:40:19.258 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:18.745 10.42.0.42 -- [02/Aug/2026:22:40:18 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:18.744 2026-08-02 22:40:18.744 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:18.232 10.42.0.42 -- [02/Aug/2026:22:40:18 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:18.231 2026-08-02 22:40:18.231 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:17.719 10.42.0.42 -- [02/Aug/2026:22:40:17 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:17.718 2026-08-02 22:40:17.718 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:17.205 10.42.0.42 -- [02/Aug/2026:22:40:17 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:17.205 2026-08-02 22:40:17.205 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:16.694 10.42.0.42 -- [02/Aug/2026:22:40:16 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:16.694 2026-08-02 22:40:16.694 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:16.188 10.42.0.42 -- [02/Aug/2026:22:40:16 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:16.179 2026-08-02 22:40:16.179 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:14.655 10.42.0.42 -- [02/Aug/2026:22:40:14 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

2026-08-03 04:10:14.654 2026-08-02 22:40:14.654 [ERROR] notes-api - Failed to fetch notes: connection to server at "notes-api-postgres" (10.43.240.40), port 5432 failed: Connection refused

2026-08-03 04:10:14.142 10.42.0.42 -- [02/Aug/2026:22:40:14 +0000] "GET /api/notes HTTP/1.1" 503 33 "-" "curl/8.5.0"

Recovery

```
ubuntu@ip-172-31-33-133:~/SRE-Project1$ # In Window 2: Ctrl+C the load loop
# In Window 1:
./tests/simulate-db-outage.sh --recover
echo "RECOVERY: $(date -u '+%Y-%m-%d %H:%M:%S UTC')"
sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -w
[RECOVERY] Scaling Postgres back to 1 replica...
deployment.apps/notes-api-postgres scaled
[RECOVERY] Waiting for Postgres to be ready...
Waiting for deployment "notes-api-postgres" rollout to finish: 0 out of 1 new replicas have been updated...
Waiting for deployment "notes-api-postgres" rollout to finish: 0 of 1 updated replicas are available...
deployment "notes-api-postgres" successfully rolled out
[RECOVERY] Done. Database should be reachable again.
RECOVERY: 2026-08-02 22:42:38 UTC
NAME                                READY   STATUS    RESTARTS   AGE
notes-api-545877b456-7965p          1/1     Running   0           3h54m
notes-api-545877b456-hqjzh          1/1     Running   0           3h54m
notes-api-postgres-5dbcdc9df5-4qsq1 1/1     Running   0           13s
```

```
503
ubuntu@ip-172-31-33-133:~$ curl -s http://13.126.63.217/api/notes
{"notes":[{"content":"hi","created_at":"2026-08-02T17:56:10.876937","id":35},{"content":"hii","created_at":"2026-08-02T17:26:21.439249","id":34},{"content":"load test note 20","created_at":"2026-08-02T17:19:43.090734","id":33},{"content":"load test note 19","created_at":"2026-08-02T17:19:43.066650","id":32},{"content":"load test note 18","created_at":"2026-08-02T17:19:43.042354","id":31},{"content":"load test note 17","created_at":"2026-08-02T17:19:43.017552","id":30},{"content":"load test note 16","created_at":"2026-08-02T17:19:42.993362","id":29},{"content":"load test note 15","created_at":"2026-08-02T17:19:42.969013","id":28},{"content":"load test note 14","created_at":"2026-08-02T17:19:42.944712","id":27},{"content":"load test note 13","created_at":"2026-08-02T17:19:42.920921","id":26},{"content":"load test note 12","created_at":"2026-08-02T17:19:42.894090","id":25},{"content":"load test note 11","created_at":"2026-08-02T17:19:42.870118","id":24},{"content":"load test note 10","created_at":"2026-08-02T17:19:42.846177","id":23},{"content":"load test note 9","created_at":"2026-08-02T17:19:42.821068","id":22},{"content":"load test note 8","created_at":"2026-08-02T17:19:42.796826","id":21},{"content":"load test note 7","created_at":"2026-08-02T17:19:42.772477","id":20},{"content":"load test note 6","created_at":"2026-08-02T17:19:42.747837","id":19},{"content":"load test note 5","created_at":"2026-08-02T17:19:42.723013","id":18},{"content":"load test note 4","created_at":"2026-08-02T17:19:42.698203","id":17},{"content":"load test note 3","created_at":"2026-08-02T17:19:42.674618","id":16},{"content":"load test note 2","created_at":"2026-08-02T17:19:42.650633","id":15},{"content":"load test note 1","created_at":"2026-08-02T17:19:42.626523","id":14},{"content":"ho","created_at":"2026-08-02T17:16:34.990946","id":13},{"content":"nt","created_at":"2026-08-02T17:06:31.094377","id":12},{"content":"14","created_at":"2026-08-02T17:06:02.245089","id":11},{"content":"13","created_at":"2026-08-02T17:04:48.711109","id":10},{"content":"13","created_at":"2026-08-02T17:02:40.648489","id":9},{"content":"12","created_at":"2026-08-02T16:57:23.850637","id":8},{"content":"hi","created_at":"2026-08-02T15:22:10.926272","id":7},{"content":"hdhhd","created_at":"2026-08-02T14:31:21.829186","id":6},{"content":"hiii","created_at":"2026-08-02T14:30:58.870060","id":5},{"content":"hellooo","created_at":"2026-08-02T13:44:40.208072","id":4},{"content":"hey test","created_at":"2026-08-02T13:44:32.407487","id":3},{"content":"My first note!","created_at":"2026-08-02T12:49:04.289890","id":2},{"content":"My first note!","created_at":"2026-08-02T12:38:37.745841","id":1}]}
```

10. Reproduction

```
ssh ubuntu@13.126.63.217
cd ~/SRE-Project1
./tests/simulate-db-outage.sh
```

```
# In a second SSH window – sustained load to keep the 5xx rate above threshold
# for the full alert-evaluation window:
for i in $(seq 1 480); do
    curl -s -o /dev/null -w "%{http_code}\n" http://13.126.63.217/api/notes
    sleep 0.5
done

# wait ~3 min, observe NotesApiHighErrorRate firing
# Ctrl+C the load loop, then recover:
./tests/simulate-db-outage.sh --recover
```

Why sustained load is needed: `NotesApiHighErrorRate` uses `rate([5m]) > 0.2`. The script's built-in one-shot burst of 15 requests would not sustain that rate across the 5-minute window. Continuous ~2 req/s load keeps the 5xx rate visible for the full alert-evaluation window and mirrors a realistic outage where traffic keeps arriving while the database is down.